Documentación del Proyecto: Sistema de Monitoreo de Calidad del Aire (IoT + IA)

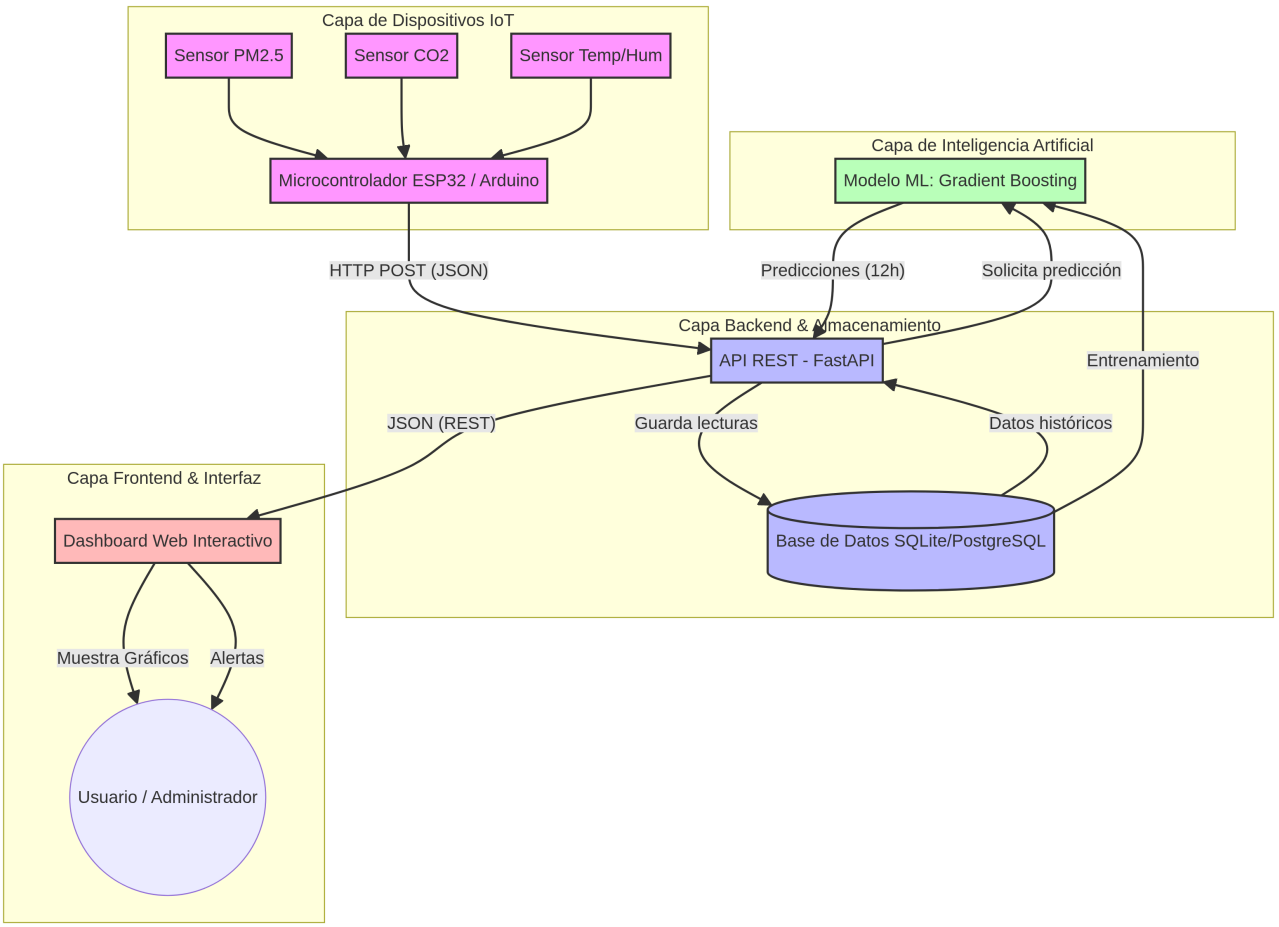
Autor: Manus AI
Fecha: Mayo 2026

1. Definición del Proyecto

Este proyecto consiste en un sistema integral para el monitoreo de la calidad del aire en tiempo real. Utiliza sensores IoT (simulados) para recolectar datos ambientales y emplea Inteligencia Artificial (IA) para predecir niveles de contaminantes (PM2.5 y CO_2). El objetivo es proveer una herramienta tecnológica que permita anticipar eventos críticos de contaminación y generar alertas para proteger la salud pública y el medio ambiente [1].

2. Arquitectura del Sistema

La arquitectura del sistema está diseñada en cuatro capas principales, asegurando modularidad y escalabilidad.



Componentes Principales:

1. **Capa de Dispositivos (IoT):** Microcontroladores (como ESP32) conectados a sensores de PM2.5, CO_2, temperatura y humedad. Recolectan datos y los envían al backend vía HTTP.
2. **Capa de Backend y Almacenamiento:** Una API REST desarrollada en Python con FastAPI. Recibe los datos y los almacena en una base de datos SQLite.
3. **Capa de Inteligencia Artificial (IA):** Un modelo de Machine Learning basado en *Gradient Boosting Regressor* (scikit-learn). Se entrena con datos históricos para predecir los niveles de contaminantes en las próximas 12 horas.
4. **Capa de Interfaz de Usuario (Frontend):** Un dashboard web interactivo (HTML/JS/CSS) que consume la API para mostrar gráficos en tiempo real, histórico de datos y alertas mediante Chart.js.

3. Desarrollo del Modelo de IA

Para realizar predicciones precisas, se ha implementado un modelo de Machine Learning utilizando la biblioteca `scikit-learn`.

3.1. Preparación de Datos

El modelo se entrena con un dataset histórico (generado por el simulador IoT) que incluye lecturas cada 15 minutos durante 30 días. Las características (*features*) extraídas incluyen:

- Variables ambientales: Temperatura y Humedad.
- Variables temporales cíclicas: Seno y coseno de la hora y el día de la semana.
- Variables categóricas: Indicador de hora pico e indicador de fin de semana.
- *Lag features*: Valores de PM2.5 y CO_2 de la lectura anterior.

3.2. Entrenamiento y Métricas

Se utiliza el algoritmo **Gradient Boosting Regressor**, el cual es robusto frente a relaciones no lineales en datos temporales. El rendimiento del modelo en la fase de prueba es el siguiente:

Contaminante	R^2 Score	RMSE
PM2.5	0.55	~3.01 µg/m^3
CO_2	0.40	~15.43 ppm

> *"El uso de modelos de ensamble como Gradient Boosting permite capturar patrones complejos en series temporales ambientales, mejorando la capacidad predictiva frente a modelos lineales simples." [2]*

4. Instrucciones de Despliegue

Requisitos Previos

- Python 3.9 o superior.
- Paquetes: `fastapi`, `uvicorn`, `scikit-learn`, `python-multipart`.

Pasos para Ejecutar el Proyecto

1. **Instalar dependencias:**

   ```
   pip install fastapi uvicorn scikit-learn python-multipart
   ```

2. **Configuración Inicial (Setup):** Ejecutar el script de setup para inicializar la base de datos, generar datos históricos y entrenar el modelo de IA.

   ```
   python backend/setup.py
   ```

3. **Iniciar el Servidor Backend:**

   ```
   python backend/app.py
   ```

   El servidor se iniciará en `http://localhost:8000`.

4. **Acceder al Dashboard:** Abrir un navegador web y navegar a `http://localhost:8000`. El dashboard cargará automáticamente la interfaz de usuario.

5. **Iniciar Simulación IoT:** Para ver datos en tiempo real, hacer una petición POST a `/api/simulation/start` o esperar a que el frontend lo inicie automáticamente.

5. Referencias

[1] Organización Mundial de la Salud (OMS). Directrices mundiales de la OMS sobre la calidad del aire.
[2] Documentación de scikit-learn: Gradient Tree Boosting. Disponible en: https://scikit-learn.org/stable/modules/ensemble.html#gradient-boosting